

TJSC 2026 - Analista de Sistemas

Segurança da Informação e Governança de TI

Aula 5

OAuth2 RFC 6749 e OpenID Connect: papéis, fluxos, scopes, claims, ID token, access token e refresh token

Campo	Definição
Disciplina	Segurança da Informação e Governança de TI
Peso no edital	8 questões no bloco de conhecimentos específicos
Recorte da aula	Protocolo OAuth2 RFC 6749 e OpenID Connect - OIDC
Objetivo	Resolver questões FGV que confundem autorização, autenticação, tokens, fluxos, scopes e claims
Perfil de estudo	Teoria objetiva, exemplos com APIs/Keycloak/Java/Quarkus/Angular, pegadinhas e 20 questões autorais comentadas

Esta aula é autoral, orientada ao edital vigente do TJSC 2026 e calibrada pelo estilo de cobrança da FGV em provas correlatas de Tecnologia da Informação. As questões são inéditas e não copiam questões reais; provas públicas da FGV foram usadas apenas para observar forma, densidade e tipo de distrator.

1. Identificação da aula e aderência ao edital

Aula pedida: Aula 5 da trilha de Segurança da Informação e Governança de TI. O recorte cobre diretamente os itens do edital: Protocolo OAuth2 RFC 6749 e OpenID Connect - OIDC. A aula também usa como apoio estritamente necessário os temas Single Sign On - SSO, Keycloak, autenticação federada, sessões e tokens, porque esses assuntos são interdependentes na prática de sistemas corporativos.

Item da trilha	Cobertura nesta aula	Limite para não fugir do edital
OAuth2 RFC 6749	Papéis, autorização delegada, authorization code, client credentials, implicit, password credentials, refresh token, scopes e tokens.	Aprofundar segurança apenas no que ajuda a resolver OAuth/OIDC em prova.
OpenID Connect - OIDC	OIDC como camada de identidade sobre OAuth2, ID Token, claims, UserInfo, discovery e validação de tokens.	Não transformar a aula em criptografia, PKI ou OWASP.
Tokens	Authorization code, access token, refresh token, ID token, bearer token e JWT quando usado.	Não supor que todo token OAuth é JWT.
Aplicação prática	Fluxos em Angular, Java/Quarkus, APIs internas, Keycloak e ambiente público/bancário.	Exemplos curtos, conectados ao padrão FGV.

2. Como a FGV tende a cobrar OAuth2 e OIDC

Em Tecnologia da Informação, a FGV costuma preferir enunciados contextualizados: um sistema público, uma API, um portal, uma integração, um servidor de identidade ou uma decisão de arquitetura. O candidato precisa identificar o conceito correto entre termos próximos. Em prova correlata recente de Cibersegurança do TJRR, por exemplo, a banca trouxe Keycloak, Single Sign On - SSO, OpenID Connect e OAuth 2.0 no mesmo contexto de afirmações V/F. Isso indica que a cobrança tende a misturar ferramenta, protocolo e finalidade.

Nesta aula, o foco é evitar as confusões que mais derrubam: OAuth2 não é, por si só, protocolo de autenticação; OpenID Connect é a camada de autenticação/identidade construída sobre OAuth2; access token autoriza acesso a recurso; ID token informa autenticação do usuário para o cliente; refresh token renova access token; scope não é sinônimo automático de cargo; claim não é sinônimo automático de permissão.

Pegadinha provável	Como a FGV pode formular	Regra de prova
OAuth2 x autenticação	Dizer que OAuth2 autentica usuário final de forma completa.	OAuth2 é autorização delegada; OIDC adiciona identidade/authenticação.
ID token x access token	Mandar a API aceitar ID token como autorização.	API deve validar access token; ID token é para o cliente/Relying Party.
Scope x role	Tratar escopo como cargo organizacional.	Scope limita acesso delegado; role pode ser mapeada, mas é outro conceito.
Bearer token	Achar que bearer prova posse de chave.	Quem porta o token pode usá-lo; por isso deve ser protegido.
Implicit e password	Apresentar como melhores práticas modernas.	São fluxos da RFC 6749, mas hoje são desaconselhados em muitos cenários.
JWT	Afirmar que todo access token deve ser JWT.	OAuth2 não exige formato; token pode ser opaco ou estruturado.

3. Conceitos-base: autorização delegada, autenticação e identidade

3.1 Autorização delegada

OAuth2 - Open Authorization 2.0 - é um **framework de autorização**. A ideia central é **permitir que uma aplicação cliente obtenha acesso limitado a um serviço HTTP sem receber diretamente a senha do usuário**. Em vez de entregar senha para cada aplicação, o **usuário interage com um servidor de autorização, que emite tokens com escopo e tempo de vida definidos**.

Exemplo corporativo: um portal web em Angular precisa consultar uma API de processos. Em vez de o Angular coletar a senha institucional e repassá-la para a API, o usuário é redirecionado para o provedor de identidade. Depois da autenticação e autorização, o cliente recebe artefatos que permitem chamar a API dentro dos limites concedidos.

3.2 Autenticação

Autenticação é provar quem é o usuário ou sistema. Login, senha, certificado, biometria, aplicativo autenticador e multifator são mecanismos de autenticação. OAuth2 puro não define como o usuário é autenticado, nem padroniza um documento de identidade do usuário para o cliente.

3.3 Identidade com OIDC

OpenID Connect - OIDC - é uma camada de identidade sobre OAuth2. Ele usa o fluxo OAuth2 e acrescenta elementos para autenticar o usuário final perante o cliente, principalmente o ID Token, um token de segurança com claims sobre a autenticação e sobre o sujeito identificado.

Pergunta	OAuth2	OpenID Connect - OIDC
Finalidade principal	Autorização delegada para acesso limitado a recursos HTTP.	Autenticação federada/identidade do usuário sobre OAuth2.
Token central	Access token; opcionalmente refresh token.	ID token, além dos tokens OAuth2.
O cliente descobre quem é o usuário?	Não de forma padronizada apenas com OAuth2.	Sim, por meio do ID token e, quando usado, UserInfo.
Exemplo de prova	Aplicativo pode acessar API X com escopo Y.	Aplicativo sabe que o usuário autenticado é o subject Z emitido pelo provedor.

4. Papéis do OAuth2 RFC 6749

A RFC 6749 define quatro papéis clássicos. Memorizar esses papéis é obrigatório, porque a FGV pode cobrá-los diretamente ou embuti-los em um caso de integração.

Papel	Tradução operacional	Exemplo no seu contexto
Resource Owner	Proprietário do recurso. Normalmente o usuário que autoriza acesso a seus dados ou ações.	Servidor/usuário que permite que o portal acesse seus dados de perfil ou processos.
Client	Aplicação que solicita acesso a recursos protegidos em nome do usuário ou em nome próprio.	Angular SPA, aplicação web, app mobile, backend Quarkus ou job de integração.
Authorization Server	Servidor que autentica o usuário quando necessário, obtém autorização e emite tokens.	Keycloak, servidor de identidade corporativo, provedor OIDC.
Resource Server	Servidor que hospeda recurso protegido e valida access token antes de responder.	API REST em Java/Quarkus que expõe processos, documentos ou pagamentos.

Pegadinha: o mesmo produto pode exercer mais de um papel, mas o papel conceitual muda conforme o fluxo. Em Keycloak, o servidor de autorização também pode atuar como provedor OIDC.

A API Quarkus normalmente é resource server. O front-end Angular é client; se ele não consegue guardar segredo com segurança, é public client.

5. Papéis e termos do OpenID Connect

No OIDC, aparecem dois nomes muito importantes: OpenID Provider - OP - e Relying Party - RP. O OP é o provedor de identidade que autentica o usuário e emite ID Token. O RP é o cliente que confia no OP para obter identidade do usuário.

Termo OIDC	Expansão	Sentido de prova
OP	OpenID Provider	Servidor que autentica o usuário final e emite ID Token. Exemplo: Keycloak.
RP	Relying Party	Cliente que confia no provedor para autenticação. Exemplo: portal web.
End-User	Usuário final	Pessoa autenticada pelo OP.
Claim	Afirmação sobre sujeito/autenticação	iss, sub, aud, exp, iat, nonce, name, email, preferred_username.
UserInfo	Endpoint de informações do usuário	Endpoint OIDC que retorna claims adicionais mediante access token válido.

6. Tokens e artefatos: o que cada um faz

6.1 Authorization code

Authorization code é um código temporário retornado ao cliente após a autorização. Ele não é o token final de acesso. O cliente troca o code no token endpoint por tokens. No fluxo Authorization Code, isso reduz a exposição de tokens no navegador, especialmente quando combinado com Proof Key for Code Exchange - PKCE.

6.2 Access token

Access token é apresentado ao resource server para acessar recurso protegido. Em APIs REST, costuma aparecer no cabeçalho HTTP Authorization: Bearer <token>. A API deve validar o token, o emissor, o público/audience, a expiração e os escopos/claims relevantes antes de permitir a operação.

6.3 Refresh token

Refresh token é usado pelo cliente para obter novo access token sem exigir nova autenticação interativa do usuário. Como costuma ter vida mais longa, precisa de armazenamento protegido, rotação, revogação e uso cuidadoso. Em prova, ele não é token para chamar API de negócio; ele é apresentado ao authorization server no token endpoint.

6.4 ID token

ID token é o token característico do OIDC. Ele contém claims sobre a autenticação do usuário final realizada pelo authorization server/OpenID Provider. Serve para o cliente verificar a identidade do usuário. Não deve ser confundido com access token usado para autorizar acesso à API.

Artefato	Quem usa	Para quê	Erro comum
Authorization code	Cliente	Trocar por tokens no token endpoint.	Tratar como access token.
Access token	Resource server/API	Autorizar acesso a recursos protegidos.	Usar como prova completa de identidade do usuário para o cliente.

Artefato	Quem usa	Para quê	Erro comum
Refresh token	Cliente no token endpoint	Obter novo access token.	Enviar para a API de negócio.
ID token	Cliente/Relying Party	Verificar autenticação e identidade do usuário.	Enviar à API como autorização.

7. Bearer token, JWT e claims

Bearer token é um token que **confere acesso a quem o apresenta**. A ideia é parecida com um **crachá ou ingresso**: se alguém indevido obtiver o token, pode tentar usá-lo até que expire ou seja revogado. Por isso, bearer token exige transporte seguro, armazenamento cuidadoso e validação rigorosa.

JSON Web Token - JWT - **é um formato compacto de token com claims em JSON**. Muitos provedores emitem ID tokens em formato JWT, e muitas arquiteturas usam JWT também para access tokens. Porém, OAuth2 não exige que o access token seja JWT; ele pode ser opaco, e a API pode validá-lo por introspecção ou por chaves públicas do emissor, conforme arquitetura.

Claim	Expansão/ideia	Uso típico
iss	Issuer - emissor	Quem emitiu o token; deve bater com o provedor esperado.
sub	Subject - sujeito	Identificador estável do usuário ou entidade autenticada.
aud	Audience - público	Para quem o token foi emitido; a API deve rejeitar token destinado a outro público.
exp	Expiration time	Data/hora de expiração; token vencido deve ser rejeitado.
iat	Issued at	Momento de emissão.
nonce	Valor aleatório associado à requisição OIDC	Ajuda a mitigar replay/injeção em fluxos de autenticação.
scope	Escopos autorizados	Permissões delegadas, como read:processos ou openid profile.

8. Scopes: permissões delegadas, não cargos mágicos

Scope define o **limite de acesso concedido ao cliente**. Em OAuth2, o cliente pede escopos; o servidor de autorização concede, reduz ou nega. Em OIDC, o escopo openid é o sinal de que o cliente está solicitando autenticação OIDC; escopos como profile, email, address e phone podem solicitar conjuntos de claims padronizados.

Em sistemas reais, escopos podem conviver com roles e permissões internas. Um usuário pode ter role ANALISTA_PROCESSUAL e o token ter scope processo:read. A role expressa posição/perfil no domínio da aplicação; o scope expressa autorização delegada concedida ao client para certo acesso. A FGV pode tentar confundir esses dois níveis.

Exemplo de chamada à API:
GET /api/processos/123
Authorization: Bearer eyJ...

Validação esperada na API:

- assinatura/chave do emissor;
- issuer esperado;
- audience da API;
- expiração;
- escopo processo:read;
- regra de negócio local, se o usuário pode ver aquele processo.

9. Fluxos OAuth2 cobrados em prova

9.1 Authorization Code Grant

Authorization Code Grant é o **fluxo clássico para aplicações web** e, hoje, também é a **base recomendada para clientes públicos quando combinado com PKCE**. O usuário interage com o authorization endpoint, o cliente recebe um code e troca esse code por tokens no token endpoint. A vantagem é não expor tokens diretamente na URL de retorno inicial.

Fluxo mental: usuário acessa cliente -> cliente redireciona para authorization server -> usuário autentica/consente -> authorization server redireciona com code -> cliente troca code por tokens -> cliente chama API com access token.

9.2 Authorization Code com PKCE

Proof Key for Code Exchange - PKCE - **protege clientes públicos contra interceptação do authorization code**. O cliente **cria um code verifier** e **envia um code challenge na solicitação inicial**. Na troca do code, **apresenta o code verifier**. O **authorization server verifica a correspondência**. Assim, interceptar apenas o code não basta para obter tokens.

9.3 Client Credentials Grant

Client Credentials Grant é usado quando o **próprio cliente acessa recursos em nome próprio, sem usuário final**. É comum em comunicação serviço-para-serviço, jobs, integrações batch, mensageria ou APIs administrativas. O resource owner, neste caso, não é uma pessoa clicando em consentimento; a autorização decorre da credencial do client e de configuração prévia.

9.4 Resource Owner Password Credentials e Implicit

Resource Owner Password Credentials - ROPC - é o fluxo em que o cliente coleta usuário e senha do resource owner para trocar por tokens. Implicit Grant retorna tokens diretamente pelo authorization endpoint. Ambos constam na RFC 6749, **mas a prática moderna os trata como problemáticos em muitos cenários**, especialmente por **exposição de credenciais ou tokens**. Para prova, memorize: existem na RFC 6749, mas não são a melhor resposta quando o enunciado pergunta por abordagem moderna e segura para SPA/mobile.

Fluxo	Quando aparece	Ponto de prova
Authorization Code	Aplicações web e fluxos com redirecionamento.	Code não é access token; é trocado no token endpoint.
Authorization Code + PKCE	SPA, mobile e clientes públicos; também boa prática moderna.	Mitiga interceptação do code; usa code verifier/challenge.
Client Credentials	Serviço-para-serviço sem usuário final.	Não emite identidade de usuário final.
ROPC	Cliente coleta senha do usuário.	Evitar como melhor prática moderna; alto acoplamento e exposição de senha.
Implicit	Tokens retornam diretamente no navegador.	Consta na RFC 6749, mas é desaconselhado para novas aplicações.
Refresh Token Grant	Renovação de access token.	Refresh token vai ao authorization server, não à API de negócio.

10. OpenID Connect em detalhe

10.1 O escopo openid

Para uma requisição ser OIDC, o cliente solicita o escopo openid. Sem esse escopo, a solicitação pode ser apenas OAuth2. Em OIDC, o retorno envolve ID token, e o cliente passa a ter uma forma padronizada de verificar quem é o usuário autenticado.

10.2 ID token

O ID token é um JWT com claims sobre a autenticação. Os pontos que mais importam em prova são: ele é destinado ao cliente/RP; contém issuer, subject, audience e expiração; deve ser validado; e não substitui o access token para chamar resource server.

10.3 UserInfo

UserInfo é um endpoint OIDC que retorna claims sobre o usuário quando recebe um access token válido. O ID token pode conter claims mínimos, e o cliente pode buscar informações adicionais no UserInfo, dependendo dos escopos concedidos e da política do provedor.

10.4 Discovery e JWKS

OIDC Discovery permite descobrir metadados do provedor, como endpoints e conjunto de chaves. JSON Web Key Set - JWKS - fornece chaves públicas usadas para validar assinaturas de tokens JWT. Em prova, esses termos aparecem para testar se o candidato entende validação descentralizada de token sem consultar senha do usuário.

11. Validação de token por uma API Java/Quarkus

Considere uma API Quarkus que expõe dados processuais. A API não deve simplesmente decodificar o JWT e confiar no conteúdo. Ela deve validar o token de forma completa. Decodificar é apenas ler o conteúdo; validar envolve verificar assinatura, emissor, público, expiração, escopos e, depois, aplicar regras de negócio.

Pseudofluxo de validação na API:

1. Receber Authorization: Bearer <access_token>.
2. Verificar se o token foi emitido pelo issuer esperado.
3. Validar assinatura com chave confiável do provedor.
4. Checar expiração e uso temporal.
5. Checar audience da API.
6. Checar scopes/claims necessários.
7. Aplicar regra de negócio local.
8. Registrar logs/auditoria sem expor o token em claro.

Pegadinha: a API não deve aceitar um token só porque ele tem nome de usuário ou e-mail. Tokens podem ser emitidos para outro público, estar vencidos, ter escopo insuficiente ou ter sido emitidos por outro realm/provedor. Em ambiente bancário ou público, esse detalhe é crítico.

12. Parâmetros e defesas importantes

Parâmetro/defesa	Uso	Confusão comum
redirect_uri	URI de retorno registrada para o cliente.	Aceitar wildcard ou domínio parecido como se fosse seguro.
state	Vincula requisição e resposta; ajuda contra Cross-Site Request Forgery - CSRF.	Confundir com nonce.
nonce	Vincula autenticação OIDC ao ID token; ajuda contra replay/injeção.	Confundir com state.
code_challenge/code_verifier	Elementos de PKCE.	Achar que PKCE substitui TLS ou validação de redirect URI.
audience - aud	Define destinatário do token.	Aceitar token emitido para outro serviço.
TLS	Transport Layer Security para proteger comunicação.	Achar que token assinado pode trafegar sem proteção.
exp	Tempo de expiração.	Usar token vencido porque a assinatura ainda é válida.

13. Exemplos de arquitetura

13.1 Angular SPA + Quarkus API

Uma aplicação Angular é um client público: o código roda no navegador e não consegue guardar segredo de client com segurança. O fluxo recomendado é Authorization Code com PKCE. O Angular redireciona o usuário ao provedor; recebe code; usa PKCE; obtém tokens; chama a API Quarkus com access token. A API valida o access token e não deve aceitar ID token como autorização.

13.2 Backend Quarkus chamando outra API

Quando um backend chama outra API sem usuário final, pode usar Client Credentials. O token representa a aplicação/serviço, não uma pessoa. Se houver necessidade de agir em nome de um usuário, o desenho muda: é preciso avaliar delegação, token exchange ou outro fluxo suportado pela plataforma, mas isso já extrapola a base do edital.

13.3 Keycloak

Em Keycloak, o realm agrupa configuração de identidade; client representa aplicação; scopes e mappers controlam claims; roles podem ser de realm ou client; e as APIs validam tokens emitidos pelo realm esperado. Para esta aula, Keycloak aparece como implementação prática de OAuth2/OIDC; a ferramenta em si foi aprofundada na Aula 4.

14. Quadro de revisão de distinções

Dupla de termos	Diferença que resolve questão
OAuth2 x OIDC	OAuth2 autoriza acesso delegado; OIDC autentica/fornece identidade sobre OAuth2.
Access token x ID token	Access token vai à API; ID token vai ao cliente para identidade/autenticação.
Scope x claim	Scope é autorização solicitada/concedida; claim é afirmação contida em token/resposta.
Role x scope	Role é papel/perfil; scope é limite de acesso delegado.
State x nonce	State protege correlação da requisição OAuth; nonce vincula autenticação OIDC ao ID token.
Authorization code x access token	Code é artefato intermediário; access token autoriza acesso ao resource server.
Public client x confidential client	Public não guarda segredo com segurança; confidential consegue proteger credencial.
JWT x bearer	JWT é formato; bearer é modo de uso em que quem possui o token pode apresentá-lo.

15. Leitura ponta a ponta de um fluxo Authorization Code com PKCE

A FGV pode cobrar esse assunto por meio de uma sequência operacional. O candidato precisa reconhecer quem chama quem, qual artefato aparece em cada etapa e qual controle de segurança está sendo aplicado. Abaixo está o roteiro mental que deve ser fixado para prova.

Etapa	O que acontece	Artefato principal	Ponto de prova
1	Cliente gera code_verifier e calcula code_challenge.	code_verifier/code_challenge e	PKCE nasce antes do redirecionamento.
2	Cliente redireciona o usuário para o authorization endpoint.	client_id, redirect_uri, scope, state, code_challenge	redirect_uri deve estar registrada e validada.

Etapa	O que acontece	Artefato principal	Ponto de prova
3	Authorization server autentica o usuário e obtém consentimento/autorização quando aplicável.	sessão do provedor	Autenticação do usuário ocorre no provedor, não na API de negócio.
4	Authorization server retorna o usuário ao redirect_uri com authorization code e state.	authorization code	Code ainda não é token de acesso.
5	Cliente envia code e code_verifier ao token endpoint.	code + code_verifier	O servidor confere se o verifier corresponde ao challenge.
6	Token endpoint emite access token e, se OIDC, ID token; refresh token depende da política e escopos.	access token, ID token, refresh token	Cada token tem destinatário e finalidade própria.
7	Cliente chama a API com access token.	Authorization: Bearer	API valida assinatura, issuer, audience, expiração, escopos e regras locais.

Observe a separação de responsabilidades: o authorization server não é a API de negócio; a API não deve autenticar usuário por senha; o cliente não deve inventar identidade com base em e-mail recebido em parâmetro de URL. A prova explora exatamente essas trocas de papel.

16. Erros de implementação que viram questão

Erro	Por que seduz	Correção
Usar ID token para chamar API	O token parece conter o usuário e a assinatura.	API deve receber access token destinado a ela.
Confiar só em decode de JWT	O payload é legível e aparenta ser verdadeiro.	Validar assinatura, issuer, audience, expiração e escopos.
Guardar client secret em SPA	Parece simplificar Client Credentials.	SPA é public client; use Authorization Code com PKCE.
Aceitar redirect_uri parcial	Facilita ambientes de homologação.	Preferir correspondência exata e URIs registradas.
Logar token completo	Ajuda debug em esteira.	Logar correlação e claims mínimos; nunca expor tokens em claro.
Tratar scope como regra total de negócio	O token trouxe autorização técnica.	Escopo é necessário, mas a API ainda aplica regra local.

17. Glossário de siglas e termos da aula

Sigla/termo	Expansão	Memorização para prova
OAuth	Open Authorization	Autorização delegada, não login completo por si só.
OIDC	OpenID Connect	Identidade/autenticação sobre OAuth2.
RFC	Request for Comments	Documento técnico normativo/informacional da IETF.
PKCE	Proof Key for Code Exchange	Proteção para Authorization Code em cliente público.
JWT	JSON Web Token	Formato de token com claims em JSON; não é sinônimo de OAuth.
JWK/JWKS	JSON Web Key / JSON Web Key Set	Chaves usadas para validar tokens assinados.
SPA	Single Page Application	Angular no navegador; normalmente public client.
API	Application Programming Interface	Resource server no cenário OAuth2.
TLS	Transport Layer Security	Protege comunicação; token assinado não dispensa TLS.
CSRF	Cross-Site Request Forgery	Ataque mitigado com state no fluxo OAuth.
OP	OpenID Provider	Provedor OIDC; autentica e emite ID token.
RP	Relying Party	Cliente que confia no OP.

18. Questões com cara de FGV: como pensar

Quando a questão mencionar um portal web, um usuário final e redirecionamento, pense em Authorization Code e, se for SPA/mobile, em PKCE. Quando mencionar integração serviço-para-serviço sem usuário, pense em Client Credentials. Quando mencionar identidade do usuário final, ID token e claims, pense em OIDC. Quando mencionar API protegida, pense em access token, audience e scopes. Quando a alternativa disser que assinatura válida basta, desconfie: geralmente ainda faltam issuer, audience, expiração e autorização.

19. Checklist do que memorizar

- RFC 6749 define OAuth2 como framework de autorização para acesso limitado a serviços HTTP.
- Papéis OAuth2: resource owner, client, authorization server e resource server.
- OIDC é camada de identidade/autenticação sobre OAuth2 e introduz ID token.
- Access token é para acessar resource server; ID token é para o cliente validar identidade do usuário.
- Refresh token renova access token no authorization server; não é token de chamada de API.
- Authorization Code com PKCE é a resposta mais segura para clientes públicos modernos.
- Client Credentials serve para comunicação serviço-para-serviço, sem usuário final.
- Scope não é sinônimo automático de role; claim não é sinônimo automático de permissão.
- Bearer token deve ser protegido: quem possui pode usar, salvo mecanismos adicionais de prova de posse.
- API deve validar assinatura, issuer, audience, expiração e escopos antes de autorizar.

20. Lista de questões autorais - padrão FGV

Instruções: resolva as 20 questões sem consultar o gabarito. As questões misturam formatos diretos, casos práticos, afirmativas, V/F, relacione e interpretação de cenário. O gabarito foi organizado de forma orgânica, sem sequência deliberada.

1. Um órgão público disponibiliza um portal de serviços digitais. Um usuário autentica-se no provedor de identidade institucional e autoriza o portal a consultar, em seu nome, uma API de processos, sem entregar sua senha ao portal. Considerando a finalidade essencial do OAuth2 descrito na RFC 6749, o mecanismo central envolvido é:

- (A) autenticação biométrica do usuário final, com emissão obrigatória de certificado digital para o navegador.
- (B) prova de identidade federada por ID token, independentemente de qualquer autorização de acesso a recursos.
- (C) autorização delegada para obtenção de acesso limitado a um serviço HTTP.
- (D) replicação de sessão entre aplicações para eliminar a necessidade de tokens.
- (E) criptografia fim a fim de todos os objetos de negócio armazenados pela API.

2. A equipe de arquitetura descreveu os seguintes elementos em uma integração OAuth2: usuário que autoriza acesso, aplicação Angular que solicita acesso, Keycloak que emite tokens e API Quarkus que hospeda o recurso protegido. Na terminologia da RFC 6749, esses elementos correspondem, respectivamente, a:

- (A) resource owner, client, authorization server e resource server.
- (B) client, resource owner, resource server e authorization server.
- (C) authorization endpoint, token endpoint, relying party e OpenID Provider.
- (D) subject, audience, issuer e claim mapper.
- (E) owner, bearer, identity provider e service registry.

3. Em uma reunião técnica, um analista afirma que OAuth2 e OpenID Connect são equivalentes, pois ambos tratam de login. Para corrigir a afirmação no nível esperado para prova, deve-se dizer que:

- (A) OAuth2 autentica o usuário final e OIDC apenas criptografa o token de acesso.
- (B) OAuth2 é específico de SAML, enquanto OIDC é específico de SOAP.
- (C) OIDC substitui completamente OAuth2 e não utiliza seus fluxos ou endpoints.
- (D) OAuth2 exige ID token em todos os fluxos, enquanto OIDC torna o ID token opcional.
- (E) OAuth2 é um framework de autorização; OIDC acrescenta uma camada de identidade/authenticação sobre OAuth2.

4. No Authorization Code Flow, uma aplicação web recebe um authorization code após o usuário autenticar-se e autorizar o acesso. A finalidade correta desse código é:

- (A) ser enviado diretamente ao resource server como prova final de autorização.
- (B) ser trocado no token endpoint por tokens, como access token e, em OIDC, ID token.
- (C) substituir o refresh token em renovações futuras de sessão.
- (D) funcionar como assinatura digital da API de negócio.
- (E) permitir que a aplicação ignore a validação de redirect_uri.

5. Uma aplicação SPA em Angular acessa APIs internas e não consegue proteger um client secret, pois seu código é entregue ao navegador. Para reduzir o risco de interceptação do authorization code, a medida mais adequada é utilizar:

- (A) Resource Owner Password Credentials, pois a senha do usuário fica disponível ao cliente.
- (B) Implicit Grant, pois evita a troca posterior no token endpoint.
- (C) Client Credentials, pois sempre representa o usuário final com maior segurança.
- (D) Authorization Code Flow com PKCE, usando code challenge e code verifier.
- (E) refresh token como parâmetro de URL no redirecionamento inicial.

6. Uma rotina batch precisa consultar uma API interna durante a madrugada, sem participação de usuário final, usando credenciais próprias da aplicação. O fluxo OAuth2 mais aderente a esse cenário é:
- (A) Implicit Grant, pois elimina a necessidade de servidor de autorização.
 - (B) Authorization Code, pois exige consentimento interativo do usuário a cada execução.
 - (C) Client Credentials Grant, pois o cliente acessa recursos em nome próprio.
 - (D) OpenID Connect Hybrid Flow, pois sempre representa pessoa física.
 - (E) Resource Owner Password Credentials, pois serviços devem guardar senha de usuário técnico em código.
7. Considere as afirmações a seguir. I. O access token é apresentado ao resource server para acessar recurso protegido. II. O ID token, no OIDC, contém claims sobre a autenticação do usuário final e é destinado ao cliente. III. O refresh token deve ser enviado a cada API de negócio como substituto do access token vencido. Está correto o que se afirma em:
- (A) I, II e III.
 - (B) I e II, apenas.
 - (C) I e III, apenas.
 - (D) II e III, apenas.
 - (E) III, apenas.
8. Um token emitido para a API de consulta processual possui escopo processo:read. O mesmo usuário também possui, no sistema interno, o papel ANALISTA. Sobre a relação entre scope e role, assinale a afirmativa correta.
- (A) Scope e role são sempre sinônimos, devendo ter o mesmo nome no token.
 - (B) Role é obrigatoriamente o mesmo que audience, enquanto scope é o issuer do token.
 - (C) Scope substitui integralmente regras de negócio locais da API.
 - (D) Role só existe em SAML, não podendo aparecer em ambientes Keycloak.
 - (E) Scope expressa autorização delegada concedida ao cliente; role pode representar perfil/papel do usuário, mas não é conceito idêntico.
9. Ao validar um token JWT recebido por uma API, a equipe confere a assinatura e verifica que o campo aud indica outro serviço, e não a API chamada. A decisão correta, sob a ótica de segurança em OAuth/OIDC, é:
- (A) rejeitar o token, pois a audience indica que ele não foi emitido para aquela API.
 - (B) aceitar o token, pois assinatura válida é suficiente em qualquer resource server.
 - (C) aceitar o token apenas se o usuário possuir e-mail institucional.
 - (D) converter o token em ID token e prosseguir.
 - (E) ignorar a audience quando o token for bearer.
10. Sobre bearer tokens no contexto OAuth2, assinale a alternativa correta.
- (A) Bearer token exige que o portador demonstre posse de chave privada a cada requisição, por definição.
 - (B) Bearer token só pode ser usado em cookies de sessão e nunca em cabeçalho HTTP.
 - (C) Bearer token é necessariamente um JWT assinado com claims de usuário.
 - (D) Quem possui o token pode apresentá-lo para acesso, razão pela qual ele deve ser protegido contra divulgação.
 - (E) Bearer token elimina a necessidade de TLS, pois o token já contém assinatura digital.
11. Em uma requisição OpenID Connect, o cliente inclui o parâmetro state e também o parâmetro nonce. A associação mais precisa é:
- (A) state protege apenas o refresh token; nonce substitui o client secret.
 - (B) state e nonce são equivalentes e um deles deve ser removido para evitar conflito.
 - (C) state ajuda a correlacionar requisição e resposta e mitigar CSRF; nonce ajuda a vincular a autenticação ao ID token e reduzir replay/injeção.
 - (D) state define a audience da API; nonce define o scope openid.

(E) state é claim obrigatória no access token; nonce é endpoint de descoberta.

12. Relacione os itens da primeira coluna às descrições da segunda. 1. ID token. 2. Access token. 3. Refresh token. 4. Authorization code. () Usado para chamar resource server. () Usado para obter novo access token junto ao authorization server. () Contém claims de autenticação do usuário para o cliente no OIDC. () Artefato temporário trocado por tokens no token endpoint. A sequência correta é:

(A) 1 - 2 - 3 - 4.

(B) 4 - 3 - 1 - 2.

(C) 2 - 1 - 3 - 4.

(D) 3 - 4 - 2 - 1.

(E) 2 - 3 - 1 - 4.

13. A respeito de fluxos previstos ou associados à RFC 6749 e às práticas modernas de segurança, assinale a alternativa correta.

(A) Implicit Grant é a opção preferencial atual para SPAs, pois impede exposição de tokens ao navegador.

(B) Resource Owner Password Credentials tende a ser problemático, pois exige que o cliente colete credenciais do usuário.

(C) Client Credentials deve ser usado quando a aplicação precisa autenticar usuário final via navegador.

(D) Authorization Code com PKCE é incompatível com clientes públicos.

(E) Refresh token deve ser enviado ao resource server em toda chamada de API.

14. Um desenvolvedor decodifica um JWT recebido em uma API e, ao encontrar o claim preferred_username, libera a operação solicitada. O principal problema dessa abordagem é:

(A) decodificar o token não equivale a validá-lo; ainda é preciso verificar assinatura, issuer, audience, expiração e autorização aplicável.

(B) claims textuais são proibidos em JWTs usados com OAuth2.

(C) preferred_username sempre identifica a senha do usuário em texto claro.

(D) APIs devem validar apenas o nome do usuário e nunca a assinatura.

(E) a presença de qualquer claim de identidade invalida automaticamente o token.

15. Em OIDC, o escopo openid tem papel especial porque:

(A) indica que o token será usado exclusivamente para criptografar banco de dados.

(B) converte o access token em refresh token permanente.

(C) define que o cliente é necessariamente confidencial.

(D) sinaliza uma requisição OpenID Connect, permitindo emissão de ID token e autenticação federada padronizada.

(E) elimina a necessidade de validação de issuer e audience.

16. Um portal público usa OIDC e precisa descobrir dinamicamente endpoints do provedor, além das chaves públicas para validar tokens assinados. Os recursos associados a essa necessidade são, respectivamente:

(A) UserInfo e refresh token rotation.

(B) authorization endpoint e client secret.

(C) OIDC Discovery e JSON Web Key Set - JWKS.

(D) resource owner password e session cookie.

(E) scope e role mapper.

17. Analise o cenário: uma API de documentos recebe um access token válido quanto à assinatura e expiração, emitido pelo provedor correto. Entretanto, o token não contém escopo para leitura de documentos. A API deve:

(A) negar a operação, pois validade estrutural do token não garante autorização suficiente para o recurso.

(B) permitir a operação, porque token válido sempre equivale a autorização total.

(C) trocar automaticamente o access token por ID token.

(D) solicitar a senha do usuário ao resource server.

(E) ignorar escopos quando o token tiver claim sub.

18. Em uma arquitetura com Angular, Keycloak e API Quarkus, assinale a opção que representa uma configuração conceitualmente adequada para cliente público no navegador.

- (A) Armazenar client secret no bundle JavaScript e usar Client Credentials para representar cada usuário.
- (B) Usar Authorization Code com PKCE, solicitar escopos necessários e enviar access token à API, que fará validação própria.
- (C) Enviar ID token à API e desabilitar a checagem de audience para simplificar a integração.
- (D) Usar ROPC para que o Angular colete diretamente a senha institucional.
- (E) Trocar refresh token diretamente com o banco de dados da aplicação.

19. Considere as seguintes afirmações sobre OIDC e claims. I. O claim sub identifica o sujeito no contexto do emissor. II. O claim iss informa o emissor do token. III. O claim exp pode ser ignorado quando a assinatura é válida. Está correto o que se afirma em:

- (A) I, II e III.
- (B) I, apenas.
- (C) II, apenas.
- (D) I e II, apenas.
- (E) II e III, apenas.

20. Assinale a alternativa que apresenta a orientação mais adequada para logs em uma API protegida por OAuth2/OIDC.

- (A) Registrar integralmente access tokens e refresh tokens em logs para facilitar auditoria futura.
- (B) Omitir qualquer informação de autenticação, inclusive identificadores de correlação e resultado de autorização.
- (C) Registrar apenas a senha do usuário, pois tokens são públicos.
- (D) Registrar tokens completos no front-end e removê-los apenas do backend.
- (E) Registrar informações mínimas úteis para auditoria, como subject, client, resultado e correlação, evitando expor tokens em claro.

21. Gabarito resumido

1-C; 2-A; 3-E; 4-B; 5-D; 6-C; 7-B; 8-E; 9-A; 10-D; 11-C; 12-E; 13-B; 14-A; 15-D; 16-C; 17-A; 18-B; 19-D;
20-E

22. Comentários completos

Questão 1 - Gabarito: C

Correta (C): A alternativa C capta a finalidade central do OAuth2: autorização delegada para acesso limitado a serviço HTTP.

Alternativa A: A autenticação biométrica pode existir no provedor, mas não define OAuth2 nem exige certificado no navegador.

Alternativa B: ID token é próprio do OIDC, não a finalidade essencial do OAuth2 puro.

Alternativa D: Replicação de sessão é tema de SSO/sessão, não o mecanismo central da RFC 6749.

Alternativa E: Criptografia de objetos pode ser controle de segurança, mas não caracteriza o OAuth2.

Questão 2 - Gabarito: A

Correta (A): A sequência correta é: resource owner (usuário), client (Angular), authorization server (Keycloak) e resource server (API).

Alternativa B: Inverte usuário e cliente e também troca os servidores.

Alternativa C: Mistura endpoints e papéis OIDC; não corresponde aos quatro papéis da RFC 6749.

Alternativa D: São claims/termos de token, não papéis centrais OAuth2.

Alternativa E: Termos genéricos e imprecisos; não são a nomenclatura da RFC 6749.

Questão 3 - Gabarito: E

Correta (E): OAuth2 trata autorização. OIDC acrescenta autenticação/identidade com ID token e claims sobre o usuário.

Alternativa A: Inverte: OAuth2 não é protocolo completo de autenticação de usuário final, e OIDC não é apenas criptografia.

Alternativa B: OAuth2/OIDC são baseados em HTTP/JSON/redirects; não são específicos de SAML ou SOAP.

Alternativa C: OIDC usa OAuth2 como base, não o substitui completamente.

Alternativa D: A presença de ID token é característica do OIDC, não obrigação de todo fluxo OAuth2.

Questão 4 - Gabarito: B

Correta (B): No Authorization Code Flow, o code é artefato intermediário trocado no token endpoint por tokens.

Alternativa A: Resource server deve receber access token, não authorization code.

Alternativa C: Refresh token tem função diferente; code não renova sessão indefinidamente.

Alternativa D: Code não é assinatura digital da API.

Alternativa E: Validação de redirect_uri continua importante; o code não dispensa esse controle.

Questão 5 - Gabarito: D

Correta (D): PKCE adiciona code challenge e code verifier, reduzindo o impacto da interceptação do authorization code em clientes públicos.

Alternativa A: ROPC força o cliente a coletar senha do usuário; não é recomendação moderna para SPA.

Alternativa B: Implicit expõe tokens no canal de redirecionamento e é desaconselhado em novas arquiteturas.

Alternativa C: Client Credentials representa o cliente, não cada usuário final em SPA.

Alternativa E: Refresh token em URL é péssima prática e não é parte do redirecionamento inicial.

Questão 6 - Gabarito: C

Correta (C): Client Credentials é adequado quando a aplicação acessa recurso em nome próprio, sem usuário final.

Alternativa A: Implicit envolve navegador/usuário e não elimina authorization server.

Alternativa B: Authorization Code pressupõe interação de usuário, inadequada para rotina batch sem usuário.

Alternativa D: Hybrid Flow/OIDC trata autenticação de usuário, não rotina em nome próprio.

Alternativa E: ROPC com senha técnica em código aumenta risco e não é o fluxo mais adequado.

Questão 7 - Gabarito: B

Correta (B): I e II estão corretas. III erra porque refresh token é usado no token endpoint para renovar access token, não em API de negócio.

Alternativa A: Inclui III, que é falsa.

Alternativa C: Inclui III e exclui II, embora ID token realmente contenha claims de autenticação para o cliente.

Alternativa D: Inclui III e exclui I, invertendo a função do access token.

Alternativa E: III isolada é justamente a afirmação errada.

Questão 8 - Gabarito: E

Correta (E): Scope limita autorização delegada; role pode indicar papel/perfil de usuário no domínio. Podem ser mapeados, mas não são idênticos.

Alternativa A: Podem se relacionar em uma implementação, mas não são sempre sinônimos.

Alternativa B: Audience e issuer são claims distintos; a frase mistura conceitos.

Alternativa C: Escopo não elimina regras de negócio locais da API.

Alternativa D: Roles podem existir em Keycloak e tokens OIDC/OAuth, conforme mapeamento.

Questão 9 - Gabarito: A

Correta (A): Audience define o destinatário do token. Se a API não é o público, deve rejeitar o token.

Alternativa B: Assinatura válida não basta; é preciso validar finalidade, emissor, expiração e escopos.

Alternativa C: E-mail institucional não resolve token destinado a outro serviço.

Alternativa D: Access token não vira ID token por conversão local.

Alternativa E: Bearer não dispensa validação de audience; pelo contrário, exige cuidado.

Questão 10 - Gabarito: D

Correta (D): Bearer token dá acesso a quem o apresenta; deve ser protegido em trânsito e armazenamento.

Alternativa A: Prova de posse de chave não é característica de bearer; isso seria PoP/mTLS/DPoP.

Alternativa B: Pode ser usado em cabeçalho Authorization, que é uso comum.

Alternativa C: Bearer token pode ser opaco ou JWT; não é necessariamente JWT.

Alternativa E: Assinatura não elimina necessidade de TLS; vazamento ainda permite uso indevido.

Questão 11 - Gabarito: C

Correta (C): State correlaciona requisição/resposta e ajuda contra CSRF. Nonce vincula autenticação OIDC ao ID token e ajuda contra replay/injeção.

Alternativa A: State não protege apenas refresh token, e nonce não substitui segredo do client.

Alternativa B: São parecidos na ideia de aleatoriedade, mas têm finalidades distintas.

Alternativa D: Audience e scope são outros parâmetros/claims.

Alternativa E: State não é claim obrigatória no access token, e nonce não é endpoint.

Questão 12 - Gabarito: E

Correta (E): Access token chama API; refresh token renova access token; ID token contém claims de autenticação; authorization code é trocado por tokens.

Alternativa A: Inverte a função dos tokens, tratando ID token como token de API.

Alternativa B: Coloca authorization code como chamada à API e access token como artefato final, invertendo parte da lógica.

Alternativa C: Acerta o primeiro item, mas coloca ID token como renovação.

Alternativa D: Começa por refresh token como chamada à API, erro clássico.

Questão 13 - Gabarito: B

Correta (B): ROPC é problemático porque o cliente coleta credenciais do usuário, aumentando acoplamento e risco.

Alternativa A: Implicit não é preferência moderna para SPA segura.

Alternativa C: Client Credentials representa o cliente/aplicação, não autenticação interativa do usuário final.

Alternativa D: Authorization Code com PKCE foi criado justamente para clientes públicos e cenários similares.

Alternativa E: Refresh token vai ao authorization server, não ao resource server.

Questão 14 - Gabarito: A

Correta (A): Ler o conteúdo não valida o token. É necessário verificar assinatura, issuer, audience, expiração e autorizações.

Alternativa B: Claims textuais não são proibidos; o problema é confiar sem validação.

Alternativa C: preferred_username não é senha em claro.

Alternativa D: Assinatura é essencial; validar apenas nome é inseguro.

Alternativa E: Claims de identidade são esperadas em ID tokens e podem existir em JWTs; não invalidam automaticamente.

Questão 15 - Gabarito: D

Correta (D): O escopo openid sinaliza uma requisição OIDC, viabilizando ID token e autenticação padronizada sobre OAuth2.

Alternativa A: Não tem relação com criptografia de banco de dados.

Alternativa B: Não converte access token em refresh token permanente.

Alternativa C: O escopo openid não define se o client é público ou confidencial.

Alternativa E: Issuer e audience continuam sendo validações relevantes.

Questão 16 - Gabarito: C

Correta (C): OIDC Discovery fornece metadados do provedor; JWKS publica chaves para validação de tokens assinados.

Alternativa A: UserInfo retorna claims do usuário; não descobre endpoints nem chaves do provedor.

Alternativa B: Authorization endpoint é um endpoint descoberto; client secret não é mecanismo de descoberta de chave pública.

Alternativa D: ROPC e cookie de sessão são temas distintos.

Alternativa E: Scope e role mapper podem afetar claims/permissões, mas não são mecanismos de descoberta e chaves.

Questão 17 - Gabarito: A

Correta (A): Token válido estruturalmente não basta; é necessário escopo/autorização suficiente para a operação.

Alternativa B: Token válido não significa autorização total.

Alternativa C: Access token não deve ser convertido em ID token pela API.

Alternativa D: Resource server não deve coletar senha do usuário.

Alternativa E: Claim sub identifica sujeito; não substitui escopo exigido.

Questão 18 - Gabarito: B

Correta (B): Para SPA, Authorization Code com PKCE é adequado; a API recebe access token e valida localmente conforme configuração.

Alternativa A: Client secret não deve ficar no JavaScript, e Client Credentials não representa cada usuário final.

Alternativa C: ID token não deve ser usado como autorização da API; audience não deve ser ignorada.

Alternativa D: ROPC faz o cliente coletar senha institucional, prática problemática.

Alternativa E: Refresh token é tratado com authorization server, não com banco de dados de aplicação.

Questão 19 - Gabarito: D

Correta (D): I e II estão corretas. III erra porque expiração deve ser validada; assinatura válida não torna token eterno.

Alternativa A: Inclui III, que é falsa.

Alternativa B: I é correta, mas II também é correta.

Alternativa C: II é correta, mas I também é correta.

Alternativa E: Inclui III, falsa, e omite I, verdadeira.

Questão 20 - Gabarito: E

Correta (E): Logs devem apoiar auditoria com dados mínimos e correlação, sem expor tokens completos em claro.

Alternativa A: Registrar tokens integrais aumenta impacto de vazamento de logs.

Alternativa B: Auditoria precisa de dados mínimos; omitir tudo prejudica rastreabilidade.

Alternativa C: Senha nunca deve ser registrada; tokens também não são públicos.

Alternativa D: Registrar tokens completos no front-end ou backend é risco relevante.

23. Referências consultadas

As referências abaixo foram usadas para validar terminologia técnica, limites da aula e estilo de cobrança. As questões são autorais e não reproduzem questões reais.

- TJSC - Edital n. 10/2026, retificado em 10/04/2026: disciplina Segurança da Informação e Governança de TI, com OAuth2 RFC 6749 e OpenID Connect - OIDC.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - prova TJRR Analista Judiciário - Cibersegurança, com item envolvendo Keycloak, SSO, OpenID Connect e OAuth 2.0.
- IETF RFC 6749 - The OAuth 2.0 Authorization Framework.
- OpenID Foundation - OpenID Connect Core 1.0.
- IETF RFC 6750 - OAuth 2.0 Bearer Token Usage.
- IETF RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients.
- IETF RFC 7519 - JSON Web Token.
- IETF RFC 9700 - Best Current Practice for OAuth 2.0 Security, usado apenas como atualização de boas práticas, sem substituir o recorte do edital.